

Design and Implementation of a 2-Digit Digital Stopwatch on FPGA

ee25btech11011 - B.Navya
ee25btech11226 - Phani Vishnu Sree

1 Introduction

This report presents the design and implementation of a 2-digit digital stopwatch using the RealDigital Boolean Board and Verilog HDL. The stopwatch counts from 00 to 59 seconds and then resets to 00. The output is shown in binary using LEDs and in decimal using the 7-segment display. The design was simulated and implemented using Xilinx Vivado.

1.1 Objectives

- Count seconds from 00 to 59 and roll over continuously.
- Display the current count in binary on 6 LEDs (`led[5:0]`).
- Display the current count in decimal (two digits) on the 7-segment display using multiplexing.
- Implement a **Start/Stop** toggle button and a **Reset** button.
- Verify correct operation through behavioural simulation.

2 System Architecture

The design is divided into a top module and two smaller modules.

- **core** – This module controls the clock division, start/stop operation, and the counter (0 to 59).
- **seg** – This module converts the count value into digits and displays it on the 7-segment display.

```
1 `timescale 1ns / 1ps
2 module top(
3     input clk,
4     input btn_start,
5     input btn_reset,
6     output [5:0] led,
7     output [6:0] seg,
8     output [3:0] an
```

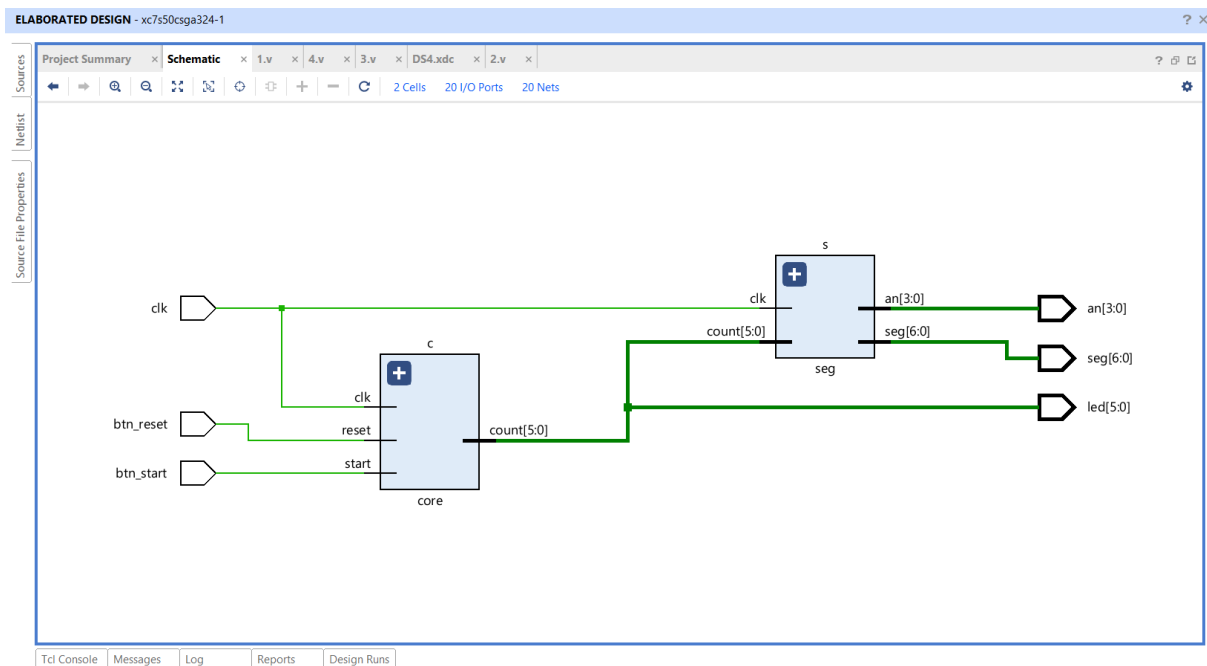
```

9 );
10 wire [5:0] count;
11
12 // core logic
13 core c( clk, btn_reset, btn_start, count );
14
15 // display
16 seg s( clk, count, seg, an);
17
18 // LEDs
19 assign led = count;
20 endmodule

```

2.1 Block Diagram

The block diagram shows the design schematic generated using Xilinx Vivado. It includes the two main modules, `core` and `seg`, and how they are connected to each other and to the top-level inputs and outputs.



2.2 Signal Flow Summary

Table 1: Top-Level Port Description

Port	Direction	Width	Description
clk	Input	1-bit	100 MHz system clock (pin F14)
btn_start	Input	1-bit	Start/Stop toggle button (pin V2)
btn_reset	Input	1-bit	Synchronous reset (pin J5)
led[5:0]	Output	6-bit	Binary count on LEDs
seg[6:0]	Output	7-bit	7-segment display segments
an[3:0]	Output	4-bit	7-segment display anodes (active-low)

3 Module Descriptions

3.1 core Module – Counter Logic

The core module implements three main functions:

```

1  'timescale 1ns / 1ps
2  module core(
3      input clk,
4      input reset,
5      input start,
6      output [5:0] count
7  );
8      // clock divider (0.1 sec)
9      reg [31:0] clk_div = 0;
10     reg tick = 0;
11     parameter MAX_COUNT = 100_000_000;
12
13     always @(posedge clk) begin
14         if (clk_div == MAX_COUNT - 1) begin
15             clk_div <= 0;
16             tick <= 1;
17         end else begin
18             clk_div <= clk_div + 1;
19             tick <= 0;
20         end
21     end
22
23     // toggle
24     reg run = 0;
25     reg prev = 0;
26     always @(posedge clk) begin
27         if (start && !prev)
28             run <= ~run;
29         prev <= start;
30     end
31
32     // counter
33     reg [5:0] cnt = 0;
34     always @(posedge clk) begin
35         if (reset)
36             cnt <= 0;
37         else if (tick && run) begin
38             if (cnt == 59)
39                 cnt <= 0;
40             else
41                 cnt <= cnt + 1;
42         end
43     end
44     assign count = cnt;
45 endmodule

```

3.1.1 Clock Divider

The board's system clock runs at 100 MHz. To obtain a 1-second tick, a 32-bit counter `clk_div` increments every clock cycle. When it reaches `MAX_COUNT -- 1` (= 99,999,999),

it resets to zero and asserts a single-cycle pulse `tick`.

$$T_{\text{tick}} = \frac{\text{MAX_COUNT}}{f_{\text{clk}}} = \frac{100,000,000}{100 \times 10^6} = 1 \text{ second}$$

3.1.2 Start/Stop Toggle

A `run` register stores the running state. Every time `btn_start` goes high and the previous value `prev` was low (rising-edge detection), `run` is toggled. This implements a single-button start/stop without debounce logic (debounce is handled by the physical button on the Boolean board).

3.1.3 Counter (0–59)

When `run` is asserted and `tick` fires, the 6-bit register `cnt` increments. When `cnt` reaches 59, it wraps back to 0. A synchronous reset (`reset`) forces `cnt` to 0 on any clock edge while `btn_reset` is high.

3.2 seg Module – 7-Segment Display Driver

The `seg` module is used to display the count value on the 7-segment display. It first separates the count into ones and tens digits using division and modulo operations.

```

1 'timescale 1ns / 1ps
2 module seg(
3     input clk,
4     input [5:0] count,
5     output reg [6:0] seg,
6     output reg [3:0] an
7 );
8
9     wire [3:0] ones = count % 10;
10    wire [3:0] tens = count / 10;
11
12    // multiplex refresh
13    reg [16:0] refresh_counter = 0;
14    reg sel = 0;
15
16    always @(posedge clk) begin
17        refresh_counter <= refresh_counter + 1;
18
19        if (refresh_counter == 100000) begin
20            refresh_counter <= 0;
21            sel <= ~sel;
22        end
23    end
24
25    reg [3:0] digit;
26    always @(*) begin
27        case(sel)
28            0: begin
29                an = 4'b1110;
30                digit = ones;
31            end
32            1: begin

```

```

33         an = 4'b1101;
34         digit = tens;
35     end
36     default: begin
37         an = 4'b1111;
38         digit = 0;
39     end
40 endcase
41 end
42
43 // decoder
44 always @(*) begin
45     case(digit)
46         4'd0: seg = 7'b1000000;
47         4'd1: seg = 7'b1111001;
48         4'd2: seg = 7'b0100100;
49         4'd3: seg = 7'b0110000;
50         4'd4: seg = 7'b0011001;
51         4'd5: seg = 7'b0010010;
52         4'd6: seg = 7'b0000010;
53         4'd7: seg = 7'b1111000;
54         4'd8: seg = 7'b0000000;
55         4'd9: seg = 7'b0010000;
56         default: seg = 7'b1111111;
57     endcase
58 end
59 endmodule

```

3.2.1 Multiplexing

The board has a 4-digit 7-segment display, but only two digits (ones and tens) are used. So, multiplexing is used to switch between the two digits very fast.

A counter is used to toggle between the digits:

- When `sel = 0`, the ones digit is displayed
- When `sel = 1`, the tens digit is displayed

Since the switching happens very fast, both digits appear to be ON at the same time to the human eye.

Table 2: Digit Selection

<code>sel</code>	<code>an[3:0]</code>	Digit
0	1110	Ones
1	1101	Tens

3.2.2 Segment Decoder

The digit value is converted into the corresponding 7-segment pattern using a `case` statement. Each number (0–9) is mapped to its display pattern.

The segments are active-low, which means:

- 0 → segment ON
- 1 → segment OFF

Table 3: 7-Segment Encoding

Digit	seg[6:0]	Hex
0	1000000	0x40
1	1111001	0x79
2	0100100	0x24
3	0110000	0x30
4	0011001	0x19
5	0010010	0x12
6	0000010	0x02
7	1111000	0x78
8	0000000	0x00
9	0010000	0x10

4 Testbench and Simulation

4.1 Testbench

A testbench (`tb_top`) is used to test the top module by giving input signals like clock, start, and reset. The clock used is 100 MHz (10 ns period).

For simulation, some values are reduced so that the output can be seen quickly:

- `MAX_COUNT` in the `core` module is changed from 100000000 to 10.
- Refresh counter in the `seg` module is changed from 100000 to 10.

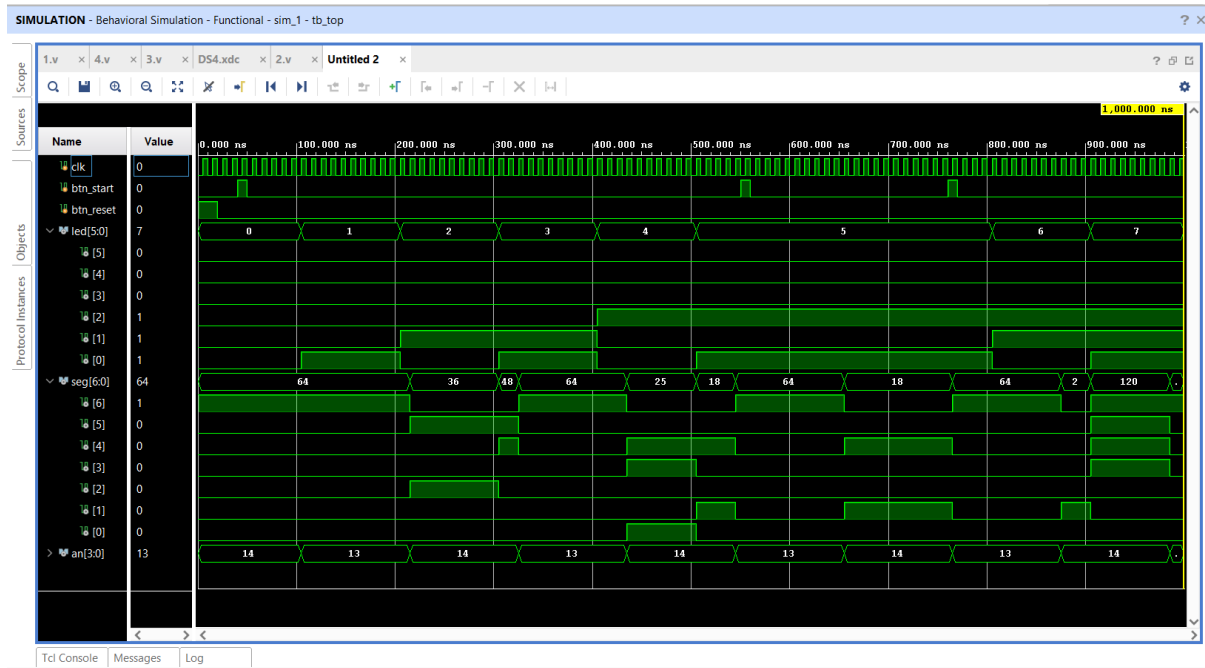
These changes are only for simulation and are changed back for hardware implementation.

```

1 'timescale 1ns / 1ps
2
3 module tb_top;
4
5     reg clk = 0;
6     reg btn_start = 0;
7     reg btn_reset = 0;
8
9     wire [5:0] led;
10    wire [6:0] seg;
11    wire [3:0] an;
12
13    // Instantiate DUT (Device Under Test)
14    top uut ( .clk(clk), .btn_start(btn_start), .btn_reset(btn_reset),
15             .led(led), .seg(seg), .an(an) );
16
17    // Clock generation (100 MHz to 10ns period)
18    always #5 clk = ~clk;

```

```
19
20     initial begin
21         // RESET
22         btn_reset = 1;
23         #20;
24         btn_reset = 0;
25
26         // START
27         #20;
28         btn_start = 1;    // press
29         #10;
30         btn_start = 0;    // release
31
32         // RUN
33         #500;
34
35         // STOP
36         btn_start = 1;
37         #10;
38         btn_start = 0;
39
40         #200;
41
42         // RESTART
43         btn_start = 1;
44         #10;
45         btn_start = 0;
46
47         #500;
48
49         //RESET AGAIN
50         btn_reset = 1;
51         #20;
52         btn_reset = 0;
53         #200;
54
55         $stop;
56     end
57 endmodule
```



5 Constraints File (XDC)

All I/O ports are assigned to physical pins on the RealDigital Boolean Board and configured with the LVCMOS33 I/O standard.

Signal	Package Pin	Description
clk	F14	100 MHz system clock
btn_reset	J5	Reset button
btn_start	V2	Start/Stop button
an[3:0]	E4, F3, J4, H3	7-segment anodes
led[5:0]	A4, B4, A3, B3, A2, B2	Binary output LEDs
seg[6:0]	D1, H4, B1, C2, D2, J3, F4	7-segment cathodes

```

1 set_property IOSTANDARD LVCMOS33 [get_ports btn_reset]
2 set_property IOSTANDARD LVCMOS33 [get_ports btn_start]
3 set_property IOSTANDARD LVCMOS33 [get_ports clk]
4 set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
5 set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
6 set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]

```



```

17 set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
21 set_property PACKAGE_PIN F14 [get_ports clk]
22 set_property PACKAGE_PIN J5 [get_ports btn_reset]
23 set_property PACKAGE_PIN V2 [get_ports btn_start]
24 set_property PACKAGE_PIN E4 [get_ports {an[3]}]
25 set_property PACKAGE_PIN F3 [get_ports {an[2]}]
26 set_property PACKAGE_PIN J4 [get_ports {an[1]}]
27 set_property PACKAGE_PIN H3 [get_ports {an[0]}]
28 set_property PACKAGE_PIN A4 [get_ports {led[5]}]
29 set_property PACKAGE_PIN B4 [get_ports {led[4]}]
30 set_property PACKAGE_PIN A3 [get_ports {led[3]}]
31 set_property PACKAGE_PIN B3 [get_ports {led[2]}]
32 set_property PACKAGE_PIN A2 [get_ports {led[1]}]
33 set_property PACKAGE_PIN B2 [get_ports {led[0]}]
34 set_property PACKAGE_PIN D1 [get_ports {seg[6]}]
35 set_property PACKAGE_PIN H4 [get_ports {seg[5]}]
36 set_property PACKAGE_PIN B1 [get_ports {seg[4]}]
37 set_property PACKAGE_PIN C2 [get_ports {seg[3]}]
38 set_property PACKAGE_PIN D2 [get_ports {seg[2]}]
39 set_property PACKAGE_PIN J3 [get_ports {seg[1]}]
40 set_property PACKAGE_PIN F4 [get_ports {seg[0]}]

```

6 Hardware Demonstration

The design was successfully implemented on the RealDigital Boolean Board. The stopwatch works as follows:

- The 7-segment display shows the time in seconds (00 to 59).
- The LEDs display the same value in binary form.
- Pressing `btn_start` starts the stopwatch, and pressing it again pauses it.
- Pressing `btn_reset` resets the count to 00.
- After reaching 59, the counter goes back to 00.

6.1 Video Demonstration

A video of the working stopwatch on the board has been uploaded to YouTube.

Demo Link: <https://youtu.be/7ZmmeEY7QVQ?si=omFyKsNPfQS1Y7BA>